

■ SC-12 — District Blood Donor Matching Platform

Complete Technical & Development Documentation

Development strategy: Backend-first • Fast MVP • Solid architecture • Parallel team development • Beautiful frontend last

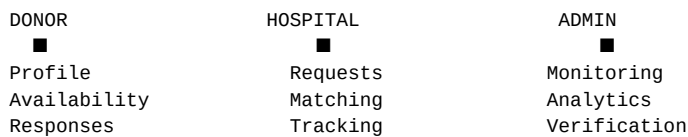
Team: 2 developers working remotely

Primary objective: Build a reliable blood-request → donor-matching → response → fulfilment workflow first, then progressively add intelligence, maps, analytics and UI polish.

1. Project Vision

Problem: During urgent blood requirements, hospitals and patients may depend on manual calls, WhatsApp groups, social networks or existing contacts. The key difficulty is finding a compatible, eligible, available donor quickly while managing location, urgency and request status.

Solution: A district-level blood donor matching platform that connects verified blood requests with suitable donors using blood-group compatibility, eligibility, availability, geographic proximity, request urgency and intelligent ranking.



2. Core Product

Donor

- Register and maintain profile
- Specify blood group and availability
- View relevant blood requests
- Accept/reject requests
- View donation history

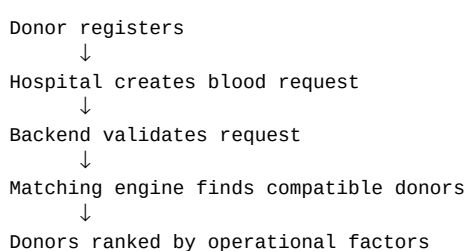
Hospital

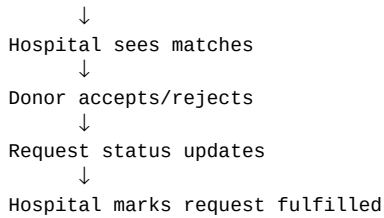
- Register/login
- Create blood requests
- Specify urgency and required units
- View matching donors
- Track donor responses
- Mark request fulfilled

Admin

- Verify donors and hospitals
- Monitor active requests
- Monitor district statistics
- Manage suspicious/inactive accounts
- View platform activity

3. MVP — Build This First



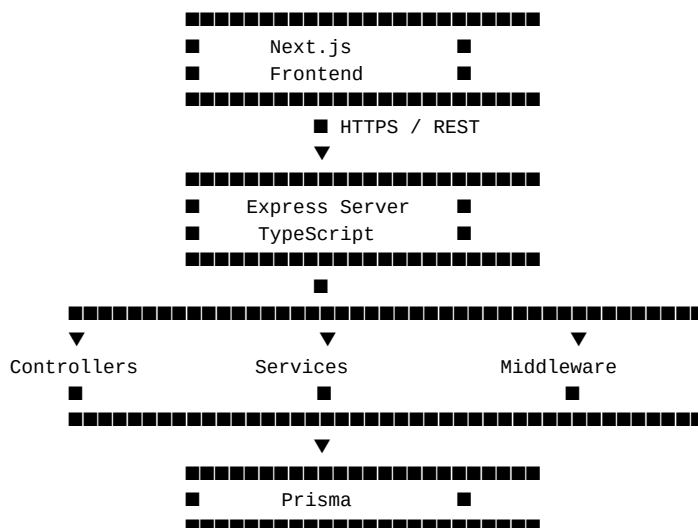


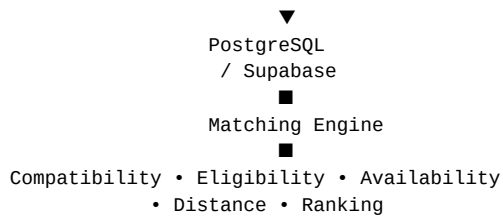
If this workflow works reliably, the core product is complete. AI, maps, analytics and advanced features are enhancements.

4. Technology Stack

Layer	Technology
Frontend	Next.js + TypeScript
UI	Tailwind CSS + shadcn/ui
Animation	Framer Motion
Backend	Node.js + Express.js + TypeScript
Validation	Zod
ORM	Prisma
Database	PostgreSQL / Supabase
Authentication	Supabase Auth
Matching	Custom TypeScript engine
Distance	Haversine initially; routing API later if needed
AI	Gemini API
Maps	Leaflet / MapLibre
Charts	Recharts
Testing	Vitest/Jest + Supertest + Postman/Bruno
API Docs	Swagger/OpenAPI
Version control	Git + GitHub
Frontend deployment	Vercel
Backend deployment	Render
Database hosting	Supabase

5. Architecture





6. Repository Structure

```

bloodbridge/
  ■■■ frontend/
  ■   ■■■ app/
  ■   ■■■ components/
  ■   ■■■ hooks/
  ■   ■■■ lib/
  ■   ■■■ types/
  ■   ■■■ public/
  ■■■ backend/
  ■   ■■■ src/
  ■   ■   ■■■ config/
  ■   ■   ■■■ controllers/
  ■   ■   ■■■ middleware/
  ■   ■   ■■■ routes/
  ■   ■   ■■■ services/
  ■   ■   ■■■ validators/
  ■   ■   ■■■ utils/
  ■   ■   ■■■ types/
  ■   ■   ■■■ app.ts
  ■   ■   ■■■ server.ts
  ■   ■■■ prisma/
  ■   ■   ■■■ schema.prisma
  ■   ■   ■■■ seed.ts
  ■   ■■■ tests/
  ■■■ docs/
  ■   ■■■ API.md
  ■   ■■■ ARCHITECTURE.md
  ■   ■■■ DATABASE.md
  ■   ■■■ MATCHING.md
  ■■■ .env.example
  ■■■ .gitignore
  ■■■ README.md
  ■■■ package.json
  
```

7. Database Design

users

id, email, name, phone, role, created_at, updated_at

donors

id, user_id, blood_group, date_of_birth, last_donation_date, latitude, longitude, availability_status, verification_status,

hospitals

id, user_id, name, address, district, latitude, longitude, verification_status, created_at, updated_at

blood_requests

id, hospital_id, blood_group, units_required, urgency, latitude, longitude, required_by, status, notes, created_at, updated_at

matches

id, request_id, donor_id, distance_km, match_score, status, notified_at, responded_at, created_at, updated_at

donation_records

id, donor_id, request_id, hospital_id, units, donated_at, created_at

User roles: DONOR, HOSPITAL, ADMIN

Availability: AVAILABLE, UNAVAILABLE, PAUSED

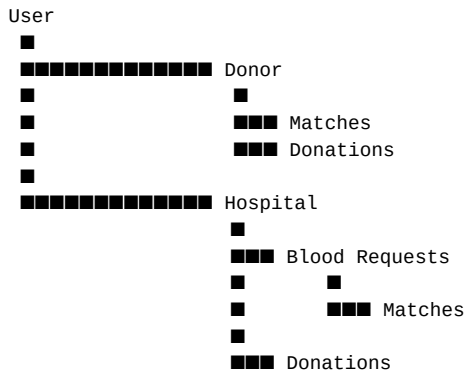
Verification: PENDING, VERIFIED, REJECTED

Urgency: NORMAL, URGENT, CRITICAL

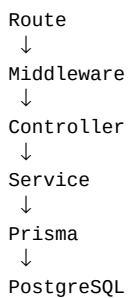
Request status: OPEN, MATCHING, PARTIALLY_FULFILLED, FULFILLED, EXPIRED, CANCELLED

Match status: PENDING, ACCEPTED, REJECTED, EXPIRED, CANCELLED

8. Database Relationships



9. Backend Architecture



Routes handle routing. Controllers handle HTTP request/response. Services contain business logic. Prisma handles database access. The matching engine stays in services rather than inside route handlers.

10. API Design

Authentication

```
POST /api/auth/register
POST /api/auth/login
POST /api/auth/logout
GET /api/auth/me
```

Donors

```
POST /api/donors
GET /api/donors/:id
PATCH /api/donors/:id
PATCH /api/donors/:id/availability
GET /api/donors/:id/requests
GET /api/donors/:id/history
```

Hospitals

```
POST /api/hospitals
GET /api/hospitals/:id
PATCH /api/hospitals/:id
```

Blood Requests

```
POST /api/requests
GET /api/requests
GET /api/requests/:id
PATCH /api/requests/:id
PATCH /api/requests/:id/status
DELETE /api/requests/:id
```

Matching

```
GET /api/requests/:id/matches
POST /api/requests/:id/match
POST /api/matches/:id/accept
POST /api/matches/:id/reject
```

Analytics — later

```
GET /api/analytics/overview
GET /api/analytics/blood-groups
GET /api/analytics/district
GET /api/analytics/requests
```

11. Validation

Every API input must be validated server-side with Zod. Validate blood group, units, urgency, coordinates, required date, IDs and user role. Never rely only on frontend validation.

12. Matching Engine

```
services/matching/
■■■ compatibility.service.ts
■■■ eligibility.service.ts
■■■ distance.service.ts
■■■ ranking.service.ts
■■■ matching.service.ts
```

```
REQUEST
↓
Blood compatibility
↓
Eligibility
↓
Availability
↓
Distance
↓
Urgency / operational ranking
↓
Ranked donors
```

Compatibility

Use a deterministic, documented compatibility rules engine. Do not ask an AI model to decide medical compatibility.

Eligibility

Filter according to the eligibility requirements defined by the challenge and authoritative health guidance. Factors can include current availability, verification status and last donation date. Medical rules must not be invented by the AI.

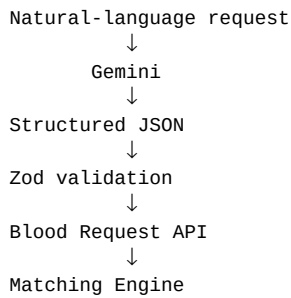
Distance

Initially calculate straight-line geographic distance using the Haversine formula. If needed later, add road-distance routing.

Ranking

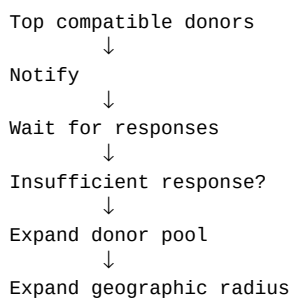
After filtering, rank suitable donors using operational factors such as distance, availability and urgency. A displayed score should be called a Priority/Match Score, not a medical probability, and its calculation should be documented.

13. AI Layer



Example: “We urgently need two units of O positive blood at the district hospital.” Gemini can extract blood group, units, urgency and location. The normal backend then validates and processes the request. If AI fails, the standard form must still work.

14. Donor Cascade — Later



This is a potential differentiating feature, but it is not required for the first MVP.

15. Frontend

Public

- Landing page
- About
- How it works
- Emergency information

Donor

- Dashboard
- Profile
- Availability
- Blood requests
- Request details

- Donation history

Hospital

- Dashboard
- Create request
- Active requests
- Matching donors
- Request tracking

Admin

- Overview
- Donors
- Hospitals
- Requests
- District analytics

16. Frontend Design Direction

Target: trustworthy + medical + modern + fast. Avoid a generic red medical website. Use clean cards, strong typography, clear status indicators, maps, data visualization, minimal animation and responsive layouts.

Blood Request
0+ • 2 Units • URGENT

[illegible]

■ Verified Donor
2.4 km
Available
Priority: High

[Request Donor]

XXXXXXXXXXXXXXXXXXXXXXXXXXXX

■ Verified Donor
4.8 km
Available

[Request Donor]

17. Remote Team Collaboration

```

GitHub Repository
  █
███
█
You           Friend
  █           █
Backend      Frontend

```

You: backend, database, APIs, matching engine, AI, testing.

Friend: UI/UX, components, dashboards, maps, charts, responsive design.

Before coding, agree on the API contract. The frontend can use mock JSON matching the contract while the backend is being built.

18. Git Workflow

```
main
■
■■■ feature/auth
■■■ feature/donor-api
■■■ feature/request-api
■■■ feature/matching
■■■ feature/donor-dashboard
■■■ feature/hospital-dashboard
```

Keep Git simple. Do not create unnecessary develop branches. Use small feature branches and pull requests.

Commit examples: feat: add donor registration • feat: implement blood request API • feat: add matching engine • fix: validate blood request • ui: add donor dashboard

19. API Contract

Create docs/API.md before both developers begin. Define endpoint, authentication requirements, request body, validation rules and response format.

```
GET /api/requests/:id/matches

{
  "requestId": "REQ001",
  "matches": [
    {
      "id": "MATCH001",
      "donor": {
        "id": "DONOR001",
        "bloodGroup": "O+"
      },
      "distanceKm": 2.4,
      "matchScore": 94,
      "availability": "AVAILABLE"
    }
  ]
}
```

20. Testing Strategy

Unit tests

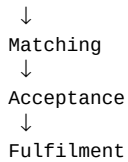
- Blood compatibility
- Eligibility
- Distance calculation
- Ranking
- Request state transitions

API tests

```
Register donor
Create request
Get matches
Accept match
Fulfill request
```

End-to-end integration

```
Donor
↓
Hospital
↓
Request
```



21. Security

- Secure authentication/session handling
- Role-based authorization
- Server-side input validation
- Rate limiting
- Correct CORS configuration
- Environment variables for secrets
- No secrets committed to GitHub
- Minimal exposure of personal information

Never commit `.env`, API keys, Supabase service keys, Gemini keys or database passwords. Maintain `.env.example`.

22. Privacy

Blood donation involves sensitive personal information. Do not publicly expose donor phone numbers, exact addresses or unnecessary medical information. Use approximate location where appropriate and reveal contact information only through an authorized workflow.

23. Seed Dataset

For development, create roughly 100–500 donors, 10–20 hospitals, multiple districts, multiple blood groups, availability states, requests and historical donations. Integrate organizer-provided data where applicable.

24. Development Stages

Stage 1 — Foundation

- GitHub repository
- Backend + frontend projects
- Supabase
- Prisma
- Environment configuration
- Database schema

Stage 2 — Core Backend

- Authentication
- Donors
- Hospitals
- Blood requests

- Request lifecycle
- Validation

Stage 3 — Matching

- Compatibility
- Eligibility
- Availability
- Haversine distance
- Ranking
- MVP complete

Stage 4 — Frontend Integration

- Donor UI
- Hospital UI
- Request creation
- Matching results
- Status tracking

Stage 5 — Intelligence

- AI request parsing
- Donor cascade
- Urgency improvements
- Analytics

Stage 6 — Visual Polish

- Maps
- Charts
- Animations
- Responsive design
- Loading/error/empty states

25. Feature Priority

■ MUST HAVE

- Authentication
- Donor profile
- Hospital profile
- Blood request
- Compatibility
- Eligibility

- Availability
- Distance
- Matching
- Accept/reject
- Request status
- Database

■ SHOULD HAVE

- Map
- Notifications
- Admin dashboard
- Analytics
- Donor history
- Verification

■ IF TIME ALLOWS

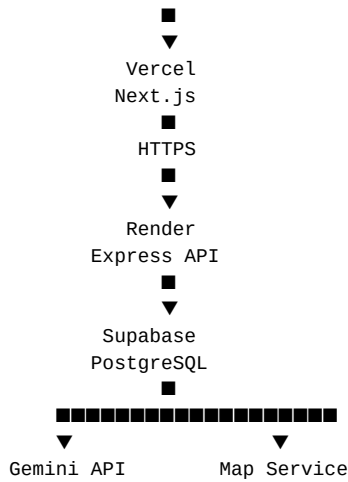
- AI assistant
- Donor cascade
- Heatmap
- Predictive analytics
- SMS
- WhatsApp
- Advanced routing
- Multi-district expansion

26. What NOT to Build

- Microservices
- Kubernetes
- Kafka
- Redis initially
- Separate Python ML server
- Complex event architecture
- Native mobile app
- Blockchain
- Custom AI model
- Over-engineered infrastructure

27. Deployment Architecture

INTERNET



28. Environment Variables

Backend

```

PORT=
DATABASE_URL=
DIRECT_URL=
SUPABASE_URL=
SUPABASE_SERVICE_ROLE_KEY=
GEMINI_API_KEY=
CORS_ORIGIN=
  
```

Frontend

```

NEXT_PUBLIC_API_URL=
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
  
```

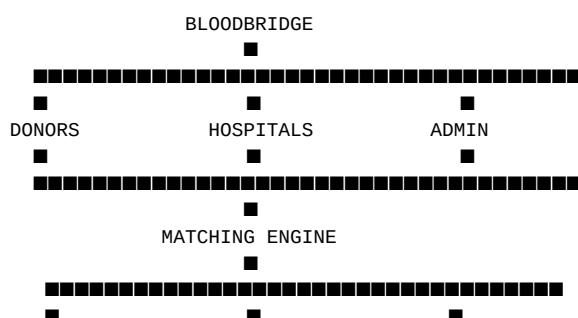
Never expose server-only secrets with NEXT_PUBLIC_.

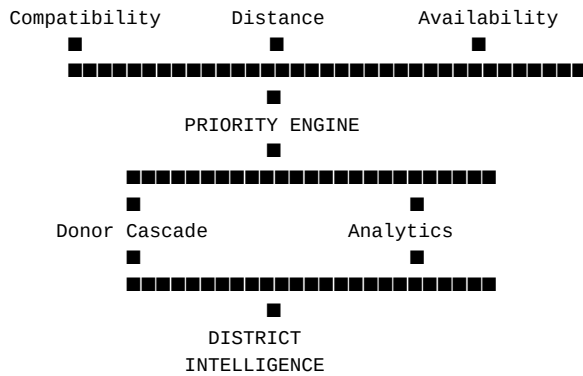
29. Complete MVP User Journey

1. Donor registers
2. Hospital creates 0+ / 2 units / URGENT request
3. Backend validates
4. Matching engine finds compatible donors
5. Distance calculated
6. Donors ranked
7. Hospital sees matches
8. Hospital sends request
9. Donor accepts
10. Second donor accepts if needed
11. Hospital marks request FULFILLED

This is the first workflow that must be bulletproof.

30. Final Product Direction





31. Exact Development Order

SESSION 1 – Foundation

GitHub → Backend → Frontend → Supabase → Prisma → Env → Schema

SESSION 2 – Core APIs

Auth → Donors → Hospitals → Blood Requests

SESSION 3 – Matching

Compatibility → Eligibility → Availability → Haversine → Ranking

SESSION 4 – Backend testing

Register → Create request → Match → Accept → Fulfill

SESSION 5 – Frontend

Donor UI → Hospital UI → Dashboards

SESSION 6 – Integration

Frontend → Backend → Database

SESSION 7 – Advanced

Maps → AI → Analytics → Cascade

FINAL

Polish → Deployment → Testing → Demo → Presentation

32. Definition of Done

The backend is considered solid when: **Given a valid blood request, the system can reliably identify eligible and compatible donors, calculate geographic proximity, rank them, record donor responses and update the request lifecycle without corrupting data.**

After that, the team can focus on making the product beautiful, adding intelligence and improving the demo.

33. Project Rules

- Backend first.
- MVP first.
- Test every critical business rule.
- Keep the API contract stable.
- Use deterministic rules for safety-critical matching.
- AI is an enhancement, never the source of truth for compatibility.
- Build frontend and backend in parallel using mock data and the API contract.
- Don't add infrastructure unless it solves a real problem.
- Keep the core workflow deployable at every stage.

- Add advanced features only after the MVP is reliable.